University of Tehran

College of Engineering
School of Electrical and Computer Engineering

Deep Generative Models Course
Instructor: Dr. Mostafa Tavassolipour

Homework Assignment 4

January 2026 (Dey 1404)

Contents

1 Question One: Diffusion Models

Diffusion Models are a new generation of generative models that have played a significant role in image, audio, and even text generation in recent years. The core idea of these models is very intuitive: we take a real data point (such as an image) and add Gaussian noise to it in several sequential steps until we reach a noise that is almost completely Gaussian. Then, we train a model to reverse this process; meaning, it can estimate the amount of added noise from the noisy data and gradually remove it to arrive back at meaningful data.

In this context, DDPM (Denoising Diffusion Probabilistic Models) [1] are one of the most famous implementations of this idea. They define a forward Markov chain (adding noise) and a reverse process (removing noise) to try to model the distribution of real data. Subsequently, the DDIM [2] model was introduced, which, by changing the sampling method in the reverse path, allows for faster sample generation and greater control over the process.

1.1 Part One: Theory Questions

Question 1: Closed Form Proof

In DDPM [1], the process of adding noise is defined as follows:

$$q(x_t|x_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}x_{t-1}, \beta_t I) \tag{1}$$

Where $\alpha_t = 1 - \beta_t$. Show that this distribution can be written for any point in time as:

$$q(x_t|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I) \tag{2}$$

Where $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$.
(4 Points)

Question 2: Advantage of Noise Prediction

In DDPM, we usually train the neural network to predict the noise ϵ at step t, rather than directly outputting the mean $\mu_\theta(x_t, t)$ or the clean image. Write down two advantages of predicting ϵ.
(4 Points)

Question 3: Cost Function Analysis (VLB)

The DDPM cost function is derived from a Variational Lower Bound (VLB) on the log-likelihood of the data and decomposes into several KL terms and a reconstruction term. In practice, training is usually done with just the simplified noise prediction loss (MSE).

(A) Conceptually, what do the KL terms in this VLB encourage?

(B) Why is it practically acceptable to ignore most of these terms and train only with the MSE noise loss?

(4 Points)

Question 4: DDIM Sampling

The DDIM (Denoising Diffusion Implicit Models) model uses the same trained DDPM model but changes the reverse process.

(A) Conceptually, how does DDIM make sampling deterministic when $\eta = 0$?

(B) Why can DDIM usually produce acceptable samples with far fewer steps than DDPM?

(4 Points)

Question 5: Classifier-Free Guidance (CFG)

Read the Classifier-Free Diffusion Guidance [3] paper and answer the following questions:

(A) What is the main idea of this method for conditioning (e.g., on text or class labels), and why is it called "Classifier-Free"?

(B) Conceptually explain what effect increasing the guidance scale has on the quality and what effect it has on the diversity of the generated samples.

(4 Points)

1.2 Part Two: Practical Questions

This section of the assignment delves into the deep investigation and step-by-step implementation of diffusion models. The goal of this section is the theoretical and practical understanding of how these models work, from basic concepts to advanced techniques like Fine-tuning.

This section includes two subsections:

- Subsection One: Implementation of basic DDPM and DDIM models.
- Subsection Two: Working with Latent Diffusion (Stable Diffusion) models and DreamBooth and LoRA techniques.

1.2.1 Subsection 1: Introduction to Diffusion Models (DDPM and DDIM)

Denoising Diffusion Probabilistic Models (DDPM) are a class of generative models that generate new data by learning to reverse a noise diffusion process.

Theory of Forward and Reverse Processes

- **Forward Process:** In this stage, Gaussian noise is gradually added to the original data (e.g., image x_0) until the image turns into complete noise x_T. This process is modeled as a Markov chain.
- **Reverse Process:** The model's goal is to learn this process; i.e., how to start from pure noise (x_T) and, by gradually removing noise, arrive at the initial image (x_0). A neural network (usually a U-Net) predicts at each stage how much noise should be removed.

Difference between DDPM and DDIM

- **DDPM:** The sampling process in DDPM is a stochastic and Markovian process. To generate a high-quality image, a large number of time steps (e.g., 1000) are usually required, which causes slow generation.
- **DDIM (Denoising Diffusion Implicit Models):** These models formulate the reverse process as deterministic and non-Markovian. This feature allows us to skip time steps, making the generation process much faster (e.g., in 50 steps instead of 1000) without a noticeable drop in quality.

Step 1: Implementing the Noise Schedule (Variance Scheduler) In the relevant cell, initialize the variable β_t linearly (Linear Scheduler). The values for β_1 and β_2 are given in the hyperparameters.
(3 Points)

Step 2: Implementing the `perturb_input` function

- This function must add noise to the image. You must do this using the "Reparameterization Trick".
- Complete the code related to calculating x_t and the noise ϵ.

(4 Points)

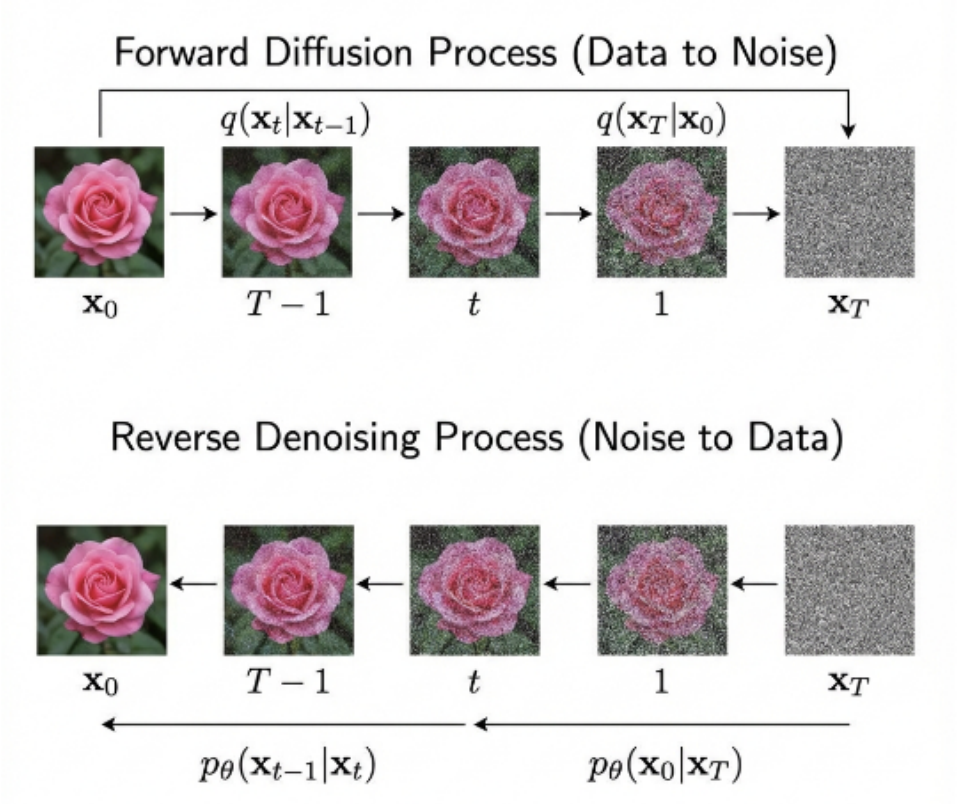

Figure 1: Diagram of Forward and Reverse Process in Diffusion Models

Step 3: Training Loop In the training loop, implement the following steps:

1. Get a batch of data from the dataloader.
2. Select a random time step t for each data point.
3. Using `perturb_input`, obtain the noisy image and the actual noise.
4. The model (neural network) predicts the noise.
5. Calculate the difference between the actual noise and the predicted noise (Loss) and Backpropagate.

(4 Points)

Step 4: Implementing Sampling

- Implement the DDPM reverse sampling algorithm.
- Start from pure noise and, in a loop from T down to 1, remove noise using the model at each stage to reach the final image.

(4 Points)

1.2.2 Subsection 2: Stable Diffusion and DreamBooth Technique

In this subsection, we get acquainted with more advanced Latent Diffusion models and learn how to train and fine-tune a large model for a specific subject.

Stable Diffusion Architecture The Stable Diffusion [4] model performs the diffusion process in the latent space rather than the pixel space to reduce computational costs. Its main components are:

- **VAE (Variational Autoencoder):** Takes the image to latent space and brings it back.
- **U-Net:** Responsible for removing noise in the latent space.
- **Text Encoder (CLIP):** Converts input text into a vector so the model understands what to generate.

LoRA and DreamBooth

- **DreamBooth [5]:** A technique to train the model with a few photos (3-5) of a specific subject (e.g., your dog). For the model to learn the subject, we bind it to a specific rare token (like `sks`) (e.g., "A photo of a sks dog").
- **Prior Preservation:** To prevent the model from forgetting (e.g., forgetting what regular dogs look like), we simultaneously feed the model generic photos generated by the model itself (Class Images).
- **LoRA [6]:** Used for fine-tuning entire models that are very heavy. LoRA freezes the model weights and only adds small, low-rank matrices to the layers that are trainable. This drastically improves training volume and speed.

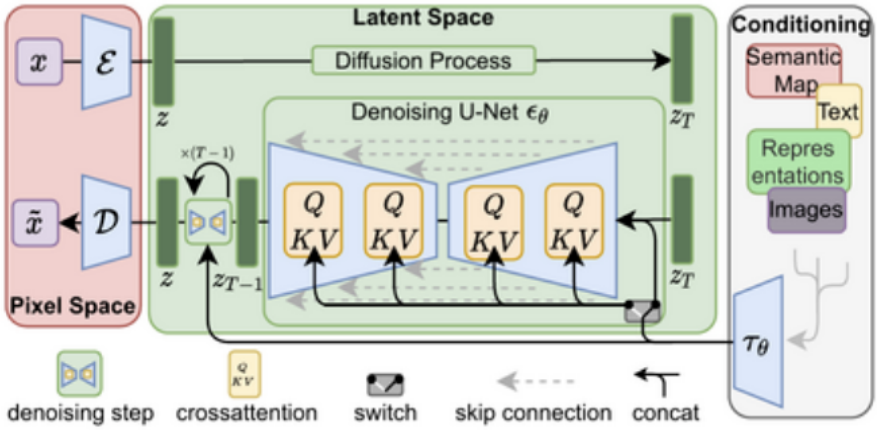

Figure 2: Stable Diffusion Architecture

Step 1: Data Preparation

- Choose a subject, prepare 5 images of it, and upload them to the `instance_data` folder.
- Set the `instance_prompt` variables (e.g., "a photo of a sks cat") and `class_prompt` (e.g., "a photo of a cat").

(2 Points)

Step 2: Completing the Dataset Class

- Complete the `__getitem__` method.
- This method must load, resize, and normalize both sample images and class images (if Prior Preservation is active). It must also tokenize the texts.

(6 Points)

Step 3: Generating Class Images (Bonus) In the bonus section, write code that uses the initial model to generate a number of generic images (e.g., 100 images of a regular dog) and save them in the `class_data` folder.

Step 4: Training Loop Inside the training loop, implement the following:

1. Generate noise and add it to the Latents (Forward process).
2. Predict noise by the model (U-Net).
3. Calculate Loss and update LoRA weights.

(7 Points)

Step 5: Test and Inference

- After training, load the model.
- Generate images with different prompts (e.g., "sks dog on the moon", "sks dog in a bucket") and different settings (`guidance_scale`) and analyze the results.

(5 Points)

2 Question Two: Flow Matching and Time Series

In recent years, Continuous Generative Models have been introduced as an efficient alternative to discrete diffusion models. One of the significant approaches in this domain is Flow Matching, which models the transition process from a simple distribution (like Gaussian noise) to the real data distribution by directly learning a Vector Field.

The goal of this assignment is a deep familiarity with the theoretical and practical foundations of Flow Matching and examining the ability of these models to model and generate real financial time series. In this assignment, Flow Matching is used to learn the distribution of Log-Returns of the stock market.

2.1 Part One: Theory Questions

Question 1: Vector Field

In Flow Matching [7] models, instead of defining a forward and reverse stochastic process, a time-dependent vector field $v_\theta(x, t)$ is trained.

(A) Explain what role this vector field plays in transporting samples from the noise distribution to the data distribution.

(B) Conceptually compare Flow Matching with Diffusion Models and state their most important difference. In what case do these two models act similarly?

(10 Points)

Question 2: Linear Paths

In Flow Matching, the intermediate data x_t is usually constructed using a linear path between noise x_1 and real data x_0.

(A) Why is using linear paths a suitable choice in training Flow Matching? Is the linear path always the optimal path? Why?

(B) What effect does this choice have on training stability and the simplicity of the cost function?

(5 Points)

Question 3: Loss Function

In the score matching method, the goal is to optimize the loss function such that it allows finding a path leading to the real data distribution. Similarly, in denoising score matching, the function $\nabla \log p(x_t)$ is used instead of the score function $\nabla \log q(x_t|x_0)$.

Assuming that:

$$q(x_t|x_0) \sim \mathcal{N}(x_0, \sigma_t^2 I) \tag{3}$$

Show that the loss function can be rewritten as follows:

$$E_{t,x_0\sim p_{data},\epsilon\sim\mathcal{N}(0,I)}\left[\left\| s_\theta(x_t, t) - \frac{x_0 - x_t}{\sigma_t^2}\right\|_2^2\right] \tag{4}$$

(5 Points)

2.2 Part Two: Practical Questions

Time series play a significant role in fields such as economics, finance, meteorology, and control systems. Financial data, in particular, possesses characteristics like high noise, temporal correlation, and non-stationary fluctuations, making them a challenging yet suitable testbed for evaluating generative models.

In this section, the goal is to use the Flow Matching model to learn the distribution of financial time series and generate realistic synthetic data.

2.2.1 Introduction to Time Series Data

In this assignment, real stock market data is used to train and evaluate the Flow Matching model. The data relates to the **SPY (S&P 500 ETF)** financial symbol, covering the time period from January 1, 2010, to December 31, 2023. The raw data includes adjusted closing prices, which, after removing missing values, are converted to Log-Returns. Using log-returns removes the price trend and makes the data more suitable for statistical modeling and machine learning.

To prepare the data, the log-return time series is converted into overlapping sequences of a fixed length of 64, so that each sample represents market behavior over a short time interval. Then, the data is globally standardized to ensure model training stability. Finally, the dataset is split into two separate parts chronologically (Chronological Split); such that the first 90% of the data is considered for training and the last 10% for testing. Using test data in the training or tuning process is not allowed and is solely used for evaluating the final model performance.

2.2.2 Implementation Notes

In implementing this assignment, you must use a Neural Network-based Flow Matching model tasked with predicting the time-dependent vector field. The model input includes the intermediate data x_t along with the time value t, and its output is a vector specifying the direction and intensity of the sample's movement in the data space. The initial noise is sampled from a standard Gaussian distribution $\mathcal{N}(0, I)$, and the time values used in training are selected uniformly from the interval $[0, 1]$.

Hyperparameter	Value
Optimizer	Adam
Batch Size	512
Learning Rate	2e-4
Epoch	100

Table 1: Table of proposed hyperparameters for model training

Proposed Hyperparameters:

2.2.3 Report of Results and Model Evaluation

In this section, report and analyze the results of training, data generation, and Flow Matching model evaluation step by step. All the following steps must be fully completed,

and the analysis related to each section must be presented in the final report.
(30 Points)

Step 1: Recording and Analyzing the Cost Function During Training During the model training process, save the cost function (Loss) value at each epoch or step. These values must be recorded for training data, and the Loss value on test data must also be calculated and saved at specified intervals.
(1 Point)

Step 2: Plotting Training and Test LOSS Charts Display the charts in a way that allows comparison between Training and Test Loss.

- Is there a significant difference between Training and Test Loss?
- If a large difference is observed, analyze the probability of overfitting or underfitting and state the probable reasons.

(3 Points)

Step 3: Synthetic Data Generation (SAMPLING) After training is complete, implement the data generation process using the trained vector field. To do this, you must start from standard Gaussian noise and generate synthetic time series by numerically solving the equation:

$$\frac{dx}{dt} = v_\theta(x, t) \tag{5}$$

in the time interval $t \in [0, 1]$. Explain what effect the number of solver steps has on the quality of the generated data and the computational cost. *Note: You can use ready-made libraries for these solvers. If used, explain the working method of these solvers.*
(15 Points)

Step 4: Displaying Generated Time Series Samples Plot a few samples of the generated time series. If possible, display both the normalized version and the version restored to the original scale (Denormalized). Check whether the amplitude, fluctuation, and general shape of the generated time series are consistent with real data.
(3 Points)

Step 5: Qualitative Comparison of Real and Generated Time Series Plot a few samples of real time series (from the test set) alongside the generated time series. Qualitatively explain to what extent the model has preserved the general patterns of the time series and whether it has weaknesses in reproducing specific behaviors (such as jumps or severe changes).
(3 Points)

Step 6: Comparing Distribution of Real and Generated Data Plot the distribution of Log-Returns values in real data and generated data using a Histogram or KDE and present your analysis.
(10 Points)

Step 7: Basic Statistical Evaluation and Volatility Calculate and report the following metrics for real and generated data:

- Mean
- Variance
- Volatility

Compare the obtained results and explain whether the model has correctly reproduced the market volatility intensity.
(5 Points)

Step 8: Advanced Structural and Temporal Distribution Evaluation In this step, the following metrics must be used:

(A) Sliced Wasserstein Distance (SWD)

(B) Autocorrelation Mean and Autocorrelation MSE

Has the model, in addition to distributional similarity, been able to correctly learn temporal dependencies? Present your analysis for each of these metrics. What does each of these metrics measure?
(10 Points)

Submission Notes

- The deadline for submitting this assignment is the end of day **Saturday, 27 Dey (January 17th)**.
- This time is not extendable, and you may use **grace time** if needed.
- Note that the maximum deadline for uploading the assignment in the system is up to 7 days after the delivery deadline, after which the system will be closed.
- Implementation must be in the **Python** programming language, and your codes must be executable and uploaded along with the report.
- This assignment is **individual**.
- In case of observing any similarity in the report or implementation codes, this will be considered **cheating** for both parties.
- Using ready-made codes without citing the source and without modification will be considered cheating, and your assignment grade will be considered zero.
- If the report format is not followed, the report grade will not be awarded to you.
- Handwritten assignment submission is not acceptable.
- All images and tables used in the report must have a caption and a number.
- A large part of your grade is related to the report and the problem-solving process.

Submission Method:
Please name and upload the files with the following format:
`HW4_[Lastname]_[StudentNumber].zip`

Contacting Teaching Assistants:
In case of questions or ambiguities, you can communicate via the following emails with the subject `DGM_HW4`:

- Question One: `mollahoseini@ut.ac.ir`
- Question Two: `fatemehnadir@gmail.com`

Wishing you health and increasing success.

References

[1] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *International Conference on Learning Representations (ICLR)*, 2021.

[3] J. Ho and T. Salimans, "Classifier-free diffusion guidance," *arXiv preprint arXiv:2207.12598*, 2022.

[4] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[5] N. Ruiz, Y. Li, V. Jampani, Y. Pritch, M. Rubinstein, and K. Aberman, "Dreambooth: Fine tuning text-to-image diffusion models for subject-driven generation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[6] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2021.

[7] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *International Conference on Learning Representations (ICLR)*, 2023.